

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 07 · LECTURE NOTES

Graphs

The data structure that is also a proof technique.

MODULE OUTCOMES

- Define graphs and their standard vocabulary.
- Prove the handshake lemma and use it.
- Explain connectivity, paths, and cycles.
- Model problems as graphs.
- Recognise bipartite and planar structure.

1 The vocabulary

- A graph is a set of vertices together with a set of edges, each edge being a pair of vertices — **directed if the pairs are ordered, weighted if the edges carry numbers**, and those two choices are independent.
- The degree of a vertex is the number of edges meeting it; in a directed graph, **in-degree and out-degree are counted separately** and their sum is the total degree.
- A walk is any sequence of adjacent vertices; a trail repeats no edge; a path repeats no vertex; a cycle is a path that returns to its start — and the distinction between these is where sloppy statements go wrong, since 'path' is used loosely in conversation and precisely in theorems.
- Connected means a path exists between every pair of vertices; a component is a maximal connected piece, and a disconnected graph is a disjoint union of its components.
- And simple means no self-loops and no repeated edges — which most theorems assume and few of them state, so it is worth checking before applying one to a multigraph or a graph with loops.

2 The handshake lemma

$$\sum_v \deg(v) = 2|E|$$

- Because each edge contributes exactly one to the degree at each of its two endpoints — so counting edge-endpoint incidences by vertex gives the left side and counting them by edge gives the right — **which is Module 06 §3's double counting in its first graph application.**
- Corollary: the number of odd-degree vertices is even. Since the total is even, the vertices contributing an odd amount must pair up — a one-line consequence with a surprising amount of reach.

- **And therefore no graph has exactly one odd-degree vertex, which rules out a whole class of proposed constructions instantly and is the usual way the lemma gets used:** somebody describes a graph with specified degrees, and you check the parity before thinking about anything else.
- **The handshake lemma is a double count — count edge-endpoint incidences by edge, and by vertex, and equate — and it is the clearest example in this course of Module 06's technique producing a genuine theorem** rather than an identity.

3 Modelling

Problem	Vertices	Edges, and what to do next
Scheduling with conflicts	Tasks or exams.	An edge for each conflict — then the problem is graph colouring.
Dependency resolution	Packages or build targets.	A directed edge for depends-on — then topologically sort, and a cycle is a circular dependency.
Routing and navigation	Locations or routers.	Weighted edges for links — then it is a shortest-path problem (CSCE 629).
Assignment and matching	Two groups: people and jobs, students and schools.	An edge for each compatible pair — then bipartite matching (CSCE 717 Module 09).
Puzzle or state search	Every reachable configuration.	An edge for each legal move — then 'is it solvable' is simply reachability. See the note.

The state-search row is the one that transfers furthest — any puzzle with states and legal moves is a graph, and 'is it solvable' becomes reachability, which is a solved problem. **Modelling is the skill; the theorems are looked up** — which is why §3 is longer than §2 in a module about graph theory.

AND THE MODELLING DECISION IS WHICH OBJECTS ARE VERTICES, WHICH IS NOT ALWAYS OBVIOUS

The same situation can be modelled in several ways, and the right choice turns a hard problem into a standard one — which is the whole art of this section, and the part that cannot be looked up.

Edge colouring a graph is vertex colouring of its line graph (vertices become edges and vice versa); a puzzle's states are vertices and its moves are edges — two reframings that convert an unfamiliar problem into one with a name and an algorithm.

So the question to ask is always: what are the objects, and what is the relation between them? — and then check whether the resulting graph has a name (bipartite, planar, acyclic, complete), because the name comes with theorems.

Which is why this module is placed before CSCE 629: that course assumes you can already turn a problem into a graph and spends its time on the algorithms instead, so the modelling step is never taught there.

4 Two special structures

BIPARTITE MEANS TWO-COLOURABLE MEANS NO ODD CYCLE — THREE EQUIVALENT STATEMENTS

A graph is bipartite if its vertices split into two sets with every edge crossing between them — equivalently, it is 2-colourable; equivalently, it contains no cycle of odd length — and proving those three equivalent is a good exercise in this module's technique.

And the equivalence hands you a test: two-colour the graph greedily by breadth-first search, and any conflict you hit exhibits an odd cycle — linear time, and the failure case is informative rather than merely negative, which is the mark of a good algorithm.

Which matters because matching problems require bipartiteness (CSCE 717 Module 09's stable matching, and the maximum-matching algorithms of CSCE 629), and a great many scheduling and assignment problems are bipartite without looking it.

Planar graphs are the other special case worth knowing — drawable in the plane without crossing edges, and satisfying Euler's formula $V - E + F = 2$ — which bounds their edge count at roughly $3V$ and is why many problems are easier on planar inputs, including some that are NP-hard in general.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapters 11 to 13 (free)	The whole module , free — graphs, colouring, and matching developed together.
MIT 6.042J, the graph theory lectures (free video)	§§1 to 4, with the modelling emphasised.
West — Introduction to Graph Theory	§§2 and 4 in depth — the standard reference when you need a theorem. Library copy.
CLRS, the graph algorithm chapters (library copy)	§3's 'what to do next' column — which is CSCE 629's content, previewed.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Define walk, trail, path, and cycle, with an example distinguishing each.
- 2 Find a theorem that assumes simple without saying so.
- 3 Prove the handshake lemma by double counting.
- 4 Prove the odd-degree corollary and use it to rule out a construction.

- 5 **Model five problems as graphs**, naming vertices and edges.
- 6 **Model one puzzle as a state graph** and compute its size.
- 7 **Find two different graph models** of the same problem, and say which is better.
- 8 **Prove bipartite and no-odd-cycle equivalent.**
- 9 **Implement the two-colouring test** and have it output the odd cycle on failure.
- 10 **Verify Euler's formula** on three planar drawings.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define graph, degree, path, cycle, connected, component, and simple.
- 2 Which assumption do most theorems make silently?
- 3 State the handshake lemma and prove it.
- 4 What is its corollary, and how is it used?
- 5 Give five problems that become graph problems, with their vertices.
- 6 Which modelling pattern transfers furthest?
- 7 What is the modelling question to ask?
- 8 Why is this module placed before CSCE 629?
- 9 Give three equivalent characterisations of bipartite.
- 10 State Euler's formula and what it bounds.